

Programación Funcional

Clases teóricas

por Pablo E. “Fidel” Martínez López

8. Inducción y recursión II

“*Aquiles:* ¿Le importaría contarme quién - o qué - es DIOS?

Genio: En absoluto, 'DIOS' es un acrónimo que significa 'DIOS que Imparte Ordenes al Subsiguiente'. Al subsiguiente Djinn, claro. La palabra árabe 'Djinn' es usada para designar Genios, Meta-Genios, Meta-Meta-Genios, etc. Es una palabra Sin Tipo.

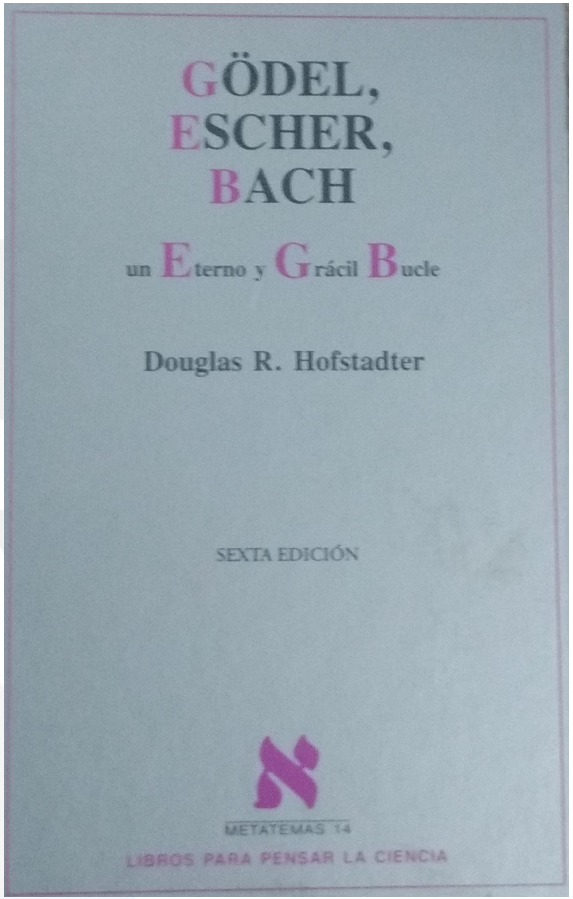
...

Aquiles: Ya veo. ¿Usted quiere decir que DIOS se sienta en la cima de la escalera de Djinn?

Genio: ¡No, no, no! No hay nada 'en la cima', pues no hay cima. Por eso DIOS es un acrónimo recursivo. DIOS no es algún último Djinn: DIOS es la torre de Djinn sobre cualquier Djinn dado.”

Gödel, Escher, Bach.
Un Eterno y Grácil Bucle.

Douglas R. Hofstadter



En el episodio anterior…

Repaso

- La ***inducción estructural*** es una *técnica* que responde las 3 preguntas planteadas
  - Permite
    - Definir tipos algebraicos con infinitos elementos
    - Definir ecuaciones orientadas que usan la función que se está definiendo del lado derecho
    - Probar propiedades sobre elementos de estos conjuntos
- Operacionalmente la llamamos ***recursión estructural***

Repaso

Definición de un conjunto **S** por ***inducción estructural***

- ***Reglas base:*** z_1 está en **S**
  ...
  z_n está en **S**
- ***Reglas inductivas:*** si $e_{11}, \dots, e_{i1}$ están en **S**, entonces e_1 está en **S**
  ...
  si $e_{1k}, \dots, e_{ik}$ están en **S**, entonces e_k está en **S**

- El *MENOR* conjunto que cumple todas las reglas (en el sentido de la inclusión)

Repaso

- Conjunto definido por ***inducción estructural***
  - También llamado ***conjunto inductivo***
    - Se define mediante *reglas básicas* y *reglas inductivas*
    - Siempre está la condición de ser el *menor* conjunto que cumple todas las reglas (en el sentido de la inclusión)
  - Las reglas pueden usarse de diversas formas
    - Como condiciones
    - Como formas de agregar elementos
    - Para verificar si un elemento es miembro del conjunto

Repaso

- Conjunto definido por ***inducción estructural***
- ¿Qué propiedades tiene?
  - Tiene infinitos elementos
  - Cada elemento satisface exactamente una regla
  - Se puede definir el orden *"es parte de"*
    - Y es **bien fundado** (no hay cadenas descententes infinitas)
  - No puede haber elementos de tamaño infinito
    - Se puede desarmar un elemento en todas sus partes y subpartes y terminar en algún momento

Repaso

- Comparar las definiciones de XBE y de BE
  - ¿Qué características comparten?
    - **Regla base 1:** △ está en XBE
    - **Regla base 2:** ▽ está en XBE
    - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
    - **Regla inductiva 2:** si $\mathbf{e_1}$ está en XBE y $\mathbf{e_2}$ está en XBE, entonces **#**$\mathbf{e_1}$**@**$\mathbf{e_2}$**//** está en XBE
    - **Regla base 1:** **T** está en BE
    - **Regla base 2:** **F** está en BE
    - **Regla inductiva 1:** si **e** está en BE, entonces **(~e)** está en BE
    - **Regla inductiva 2:** si $\mathbf{e_1}$ está en BE y $\mathbf{e_2}$ está en BE, entonces **(**$\mathbf{e_1}$**^**$\mathbf{e_2}$**)** está en BE

Repaso

- Comparar las definiciones de XBE y de BE
  - ¿Qué características comparten?
    - Misma cantidad de reglas base y misma cantidad de reglas inductivas con la misma cantidad de antecedentes cada una
  - ¿Qué implica esto?
    - Los conjuntos tienen la misma cantidad de elementos
    - Los órdenes “es parte de” de ambos son similares
  - Los conjuntos tienen la misma ***estructura***
    - Los símbolos particulares NO son importantes

Repaso

Definición de una función por ***recursión estructural***

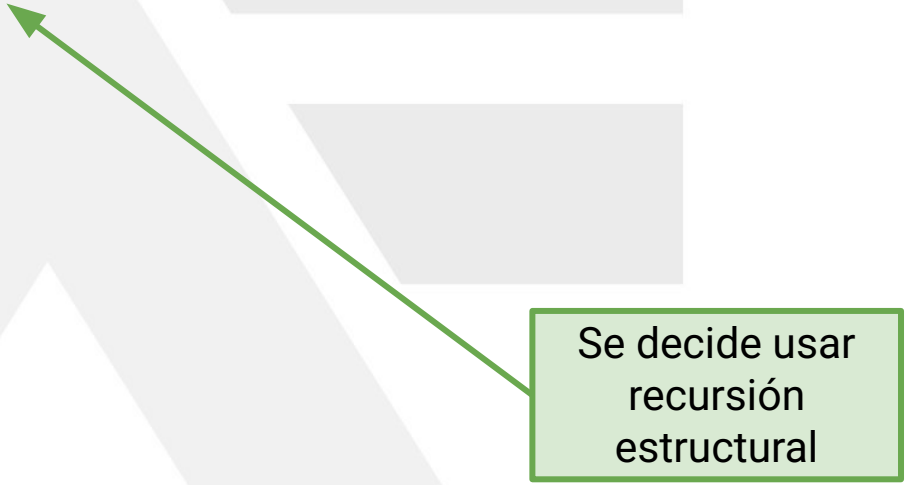
- Por cada **elemento base**, dar el resultado directamente
- Por cada **elemento inductivo**, dar el resultado usando el valor de transformar las *partes* con la misma *función que se define*

Definición de **f : : S -> T** por ***recursión estructural***

```
f :: S -> T
f z1 = ...
...
f zn = ...
f e1 = ... f e11 ... f ei1 ...
...
f ek = ... f e1k ... f eik ...
```

Repaso

- Definir `nhash,nbar :: XBE -> Int`, el número de # y el de /
  - Por recursión estructural



Repaso

- Definición *inductiva* de XBE
  - **Regla base 1:** △ está en XBE
  - **Regla base 2:** ▽ está en XBE
  - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
  - **Regla inductiva 2:** si e_1 está en XBE y e_2 está en XBE, entonces **#e_1@e_2//** está en XBE

- Definir **`nhash,nbar :: XBE ->`**
  - Por recursión estructural

```
nhash :: XBE -> Int
nhash △          = ...
nhash ▽          = ...
nhash #$e//      = ... nhash e ...
nhash #e1@e2//   = ... nhash e1 ... nhash e2 ...

nbar :: XBE -> Int
nbar △           = ...
nbar ▽           = ...
nbar #$e//       = ... nbar e ...
nbar #e1@e2//    = ... nbar e1 ... nbar e2 ...
```

Se plantea la estructura en base a las reglas

Repaso

- Definición *inductiva* de XBE
  - **Regla base 1:** △ está en XBE
  - **Regla base 2:** ▽ está en XBE
  - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
  - **Regla inductiva 2:** si e_1 está en XBE y e_2 está en XBE, entonces **#e_1@e_2//** está en XBE

- Definir **`nhash,nbar :: XBE ->`**
  - Por recursión estructural

```
nhash :: XBE -> Int
nhash △          = ...
nhash ▽          = ...
nhash #$e//      = 1 + nhash e
nhash #e1@e2//   = 1 + nhash e1 + nhash e2

nbar :: XBE -> Int
nbar △           = ...
nbar ▽           = ...
nbar #$e//       = 2 + nbar e
nbar #e1@e2//    = 2 + nbar e1 + nbar e2
```

Se definen los casos inductivos

Repaso

- Definición *inductiva* de XBE
  - **Regla base 1:** △ está en XBE
  - **Regla base 2:** ▽ está en XBE
  - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
  - **Regla inductiva 2:** si $\mathbf{e_1}$ está en XBE y $\mathbf{e_2}$ está en XBE, entonces **#e_1@e_2//** está en XBE

- Definir **`nhash,nbar :: XBE ->`**
  - Por recursión estructural

```
nhash :: XBE -> Int
nhash △          = 0
nhash ▽          = 0
nhash #$e//      = 1 + nhash e
nhash #e1@e2//   = 1 + nhash e1 + nhash e2

nbar :: XBE -> Int
nbar △           = 0
nbar ▽           = 0
nbar #$e//       = 2 + nbar e
nbar #e1@e2//    = 2 + nbar e1 + nbar e2
```

Se completa con los casos base

Repaso

- ¿Y por qué funciona la *recursión estructural*?
  - Si usa la misma función, ¿por qué termina (y no da $\bot$)?
  - ¡El orden **“es parte de”** es *bien fundado*!
    - O sea, nunca se puede descomponer un elemento en sus partes de forma infinita…
    - De esta forma, la reducción *debe terminar*
  - La condición de ser el ***menor*** es esencial para esto
  - **Una función recursiva estructural *termina* de reducir para todos los elementos inductivos**

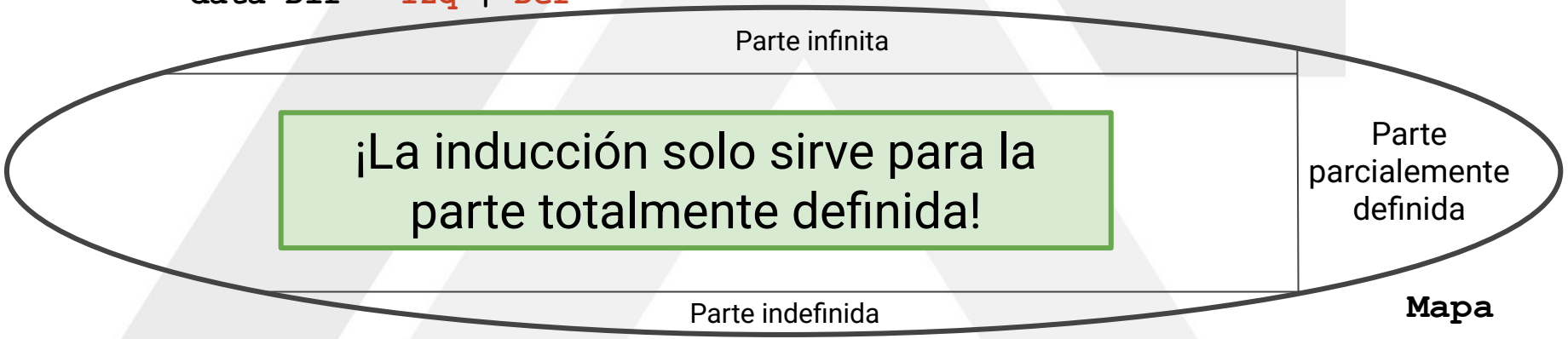
Repaso

- Definición *inductiva* de $\mathbb{N}$
  - **Regla base:** 0 está en $\mathbb{N}$
  - **Regla inductiva:** si **n** está en $\mathbb{N}$, entonces **n+1** está en $\mathbb{N}$
- ```
  f :: ℕ -> T
  f  0 = ...
  f  n = ... f  (n-1) ...
  ```
- ¡Los números naturales son un conjunto inductivo estructural!
- Funciones como **`factorial`** funcionan por su estructura
  ```

Repaso

- ¿Cómo usar todas estas técnicas en Haskell?
  - ¿Definición inductiva de estructuras?

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa
data Cofre = Monedas Int | CorazonDe Pirata | Joyas Int | Nada
data Pirata = DavyJones | WillTurner
data Dir = Izq | Der
```



Repaso

- ¿Cómo usar todas estas técnicas en Haskell?
  - ¿Definición de funciones por recursión estructural?

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn :: Mapa -> Cofre
tesoroEn (Equis c)   = dar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarDar d (tesoroEn m)

dar c           = c   -- Implementaciones triviales
caminarYDar n c = c   -- Podrían complicarse con un
girarYDar d c   = c   --  terreno y verificaciones…
```

Repaso

- ¿Cómo usar todas estas técnicas en Haskell?
  - ¿Definición de funciones por recursión estructural?

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

comprimido :: Mapa -> Mapa -- Comprime tramos rectos consecutivos
comprimido (Equis c)    = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)   = Giro d (comprimido m)

compRecto n (Recto n' m) = Recto (n+n') m
compRecto n m            = Recto n m
```

Principio de inducción estructural

Principio de inducción estructural

- ¿Cómo aprovechar la estructura en demostraciones?

Principio de inducción estructural

- ¿Cómo aprovechar la estructura en demostraciones?
  - Expresando un principio de demostración que la use

Principio de inducción estructural

- ¿Cómo aprovechar la estructura en demostraciones?
  - Expresando un principio de demostración que la use
    - Por cada regla base, un caso directo
    - Por cada regla inductiva, un caso que utilice las partes

Principio de inducción estructural

- ¿Cómo aprovechar la estructura en demostraciones?
  - Expresando un principio de demostración que la use
    - Por cada regla base, un caso directo
      - **Casos base**
    - Por cada regla inductiva, un caso que utilice las partes
      - **Casos inductivos**
        - **Hipótesis inductivas** (una por cada antecedente)
        - **Tesis inductiva**

Principio de inducción estructural

- ¿Cómo aprovechar la estructura en demostraciones?
  - Expresando un principio de demostración que la use
    - Por cada regla base, un caso directo
      - **Casos base**
    - Por cada regla inductiva, un caso que utilice las partes
      - **Casos inductivos**
        - **Hipótesis inductivas** (una por cada antecedente)
        - **Tesis inductiva**
  - Una propiedad sobre todo el conjunto resulta así equivalente a una serie de otras propiedades más restringidas

Principio de inducción estructural

- Definir **`nhash,nbar :: XBE -> Int`**, el número de # y el de /
  - Por recursión estructural

```
nhash :: XBE -> Int
nhash △          = 0
nhash ▽          = 0
nhash #$e//      = 1 + nhash e
nhash #e1@e2//   = 1 + nhash e1 + nhash e2

nbar :: XBE -> Int
nbar △           = 0
nbar ▽           = 0
nbar #$e//       = 2 + nbar e
nbar #e1@e2//    = 2 + nbar e1 + nbar e2
```

Principio de inducción estructural

- **Prop.:** ¿`nbar x = 2*nhash x`, para todo `x :: XBE`?

  Dem.:

Principio de inducción estructural

- **Prop.:** ¿`nbar x = 2*nhash x`, para todo `x : :` XBE?

  Dem.: sea **e** un elemento cualquiera de **XBE**

  Caso **e** = △) ¿`nbar △ = 2*nhash △`?

  Caso **e** = ▽) ¿`nbar ▽ = 2*nhash ▽`?

  Caso **e** = #$e'//) ¿`nbar #$e'//= 2*nhash #$e'//`?

  Caso **e** = $\#e_1@e_2//$) ¿`nbar` $\#e_1@e_2// =$ `2*nhash` $\#e_1@e_2//$?

  - Se plantean los casos
  - Y luego se procede a la demostración

Principio de inducción estructural

- **Prop.:** ¿`nbar x = 2*nhash x`, para todo `x : :` XBE?

  Dem.: sea **e** un elemento cualquiera de **XBE**

  Caso **e** = △) ¿`nbar △ = 2*nhash △`?

  `nbar △`
  = (`nbar.1`)
  `0`

  `2*nhash △`
  = (`nhash.1`)
  `2*0`
  = (aritm.)
  `0`
  Vale este caso

  Caso **e** = ▽) ¿`nbar ▽ = 2*nhash ▽`?

  `nbar ▽`
  = (`nbar.2`)
  `0`

  `2*nhash ▽`
  = (`nhash.2`)
  `2*0`
  = (aritm.)
  `0`
  Vale este caso

Principio de inducción estructural

- **Prop.:** ¿`nbar x = 2*nhash x`, para todo `x : :` XBE?

  Dem.: sea **e** un elemento cualquiera de **XBE**

  Caso **e** = #$e'//) ¿`nbar #$e'// = 2*nhash #$e'//`?

```
  nbar #$e'//
=                                  (nbar.3)
  2 + nbar e'
```

```
  2*nhash #$e'//
=                                  (nhash.3)
  2*(1 + nhash e')
=                                  (aritm.)
  2 + 2*nhash e'
```

Principio de inducción estructural

- **Prop.:** ¿`nbar x = 2*nhash x`, para todo `x : :` XBE?

  Dem.: sea **e** un elemento cualquiera de **XBE**

  Caso **e** = #$e'//) ¿`nbar #$e'// = 2*nhash #$e'//`?

$$\begin{array}{l} \texttt{nbar \#\$e'//} \\ = \\ \texttt{2 + nbar e'} \end{array} \quad \begin{array}{l} \\ \\ (\texttt{nbar.3}) \end{array}$$

$$\begin{array}{l} \texttt{2*nhash \#\$e'//} \\ = \\ \texttt{2*(1 + nhash e')} \\ = \\ \texttt{2 + 2*nhash e'} \end{array} \quad \begin{array}{l} \\ (\texttt{nhash.3}) \\ \\ (\text{aritm.}) \\ \\ \end{array}$$

  - Falta algo…
  - Es necesaria información sobre **e'**

Principio de inducción estructural

- **Prop.:** ¿`nbar x = 2*nhash x`, para todo `x : :` XBE?

  Dem.: sea **e** un elemento cualquiera de **XBE**

  Caso **e** = #e_1@e_2//) ¿`nbar` #e_1@e_2// = `2*nhash` #e_1@e_2//?

$$\texttt{nbar}\ \#e_1@e_2// \overset{(\texttt{nbar.4})}{=} 2 + \texttt{nbar}\ e_1 + \texttt{nbar}\ e_2$$

$$\texttt{2*nhash}\ \#e_1@e_2// \overset{(\texttt{nhash.2})}{=} \texttt{2*(1 + nhash}\ e_1 + \texttt{nhash}\ e_2\texttt{)} \overset{(\text{aritm.})}{=} 2 + \texttt{2*nhash}\ e_1 + \texttt{2*nhash}\ e_2$$

Principio de inducción estructural

- **Prop.:** ¿`nbar x = 2*nhash x`, para todo **x** : : XBE?

  Dem.: sea **e** un elemento cualquiera de **XBE**

  Caso **e** = #e_1@e_2//) ¿`nbar` #e_1@e_2// = `2*nhash` #e_1@e_2//?

$$\texttt{nbar}\ \#e_1@e_2// \overset{(\texttt{nbar.4})}{=} 2 + \texttt{nbar}\ e_1 + \texttt{nbar}\ e_2$$

$$\texttt{2*nhash}\ \#e_1@e_2// \overset{(\texttt{nhash.2})}{=} \texttt{2*(1 + nhash}\ e_1 + \texttt{nhash}\ e_2\texttt{)} \overset{(\text{aritm.})}{=} 2 + \texttt{2*nhash}\ e_1 + \texttt{2*nhash}\ e_2$$

  - Falta algo…
  - Es necesaria información sobre e_1 y e_2

Principio de inducción estructural

Principio de inducción estructural

- Por cada **elemento base**, demostrar que la propiedad vale en él
- Por cada **elemento inductivo**, demostrar que la propiedad vale en él, *suponiendo válida la propiedad en las partes inductivas (directas)*

Principio de inducción estructural

Principio de inducción estructural

¿para todo **x :: S. P(x)**?

es equivalente a

Caso base 1) ¿$P(z_1)$?

...

Caso base n) ¿$P(z_n)$?

Caso inductivo 1) HI_{11}) ¡$P(e_{11})$! … HI_{i1}) ¡$P(e_{i1})$!
TI_1) ¿$P(e_1)$?

...

Caso inductivo k) HI_{1k}) ¡$P(e_{1k})$! … HI_{ik}) ¡$P(e_{ik})$!
TI_k) ¿$P(e_k)$?

Principio de inducción es

- Definición *inductiva* de XBE
  - **Regla base 1:** △ está en XBE
  - **Regla base 2:** ▽ está en XBE
  - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
  - **Regla inductiva 2:** si e_1 está en XBE y e_2 está en XBE, entonces **#e_1@e_2//** está en XBE

- **Prop.:** **¿nbar x = 2*nhash x**

  Dem.: sea **e** un elemento de **XBE**. Por ppio. de ind. en la estructura de **e : : XBE**

  Caso base 1, **e = △**) **¿nbar △ = 2*nhash △?**

  Caso base 2, **e = ▽**) **¿nbar ▽ = 2*nhash ▽?**

  Caso ind. 1, **e = #$e'//**) **HI) ¡nbar e' = 2*nhash e'!**
  TI) ¿nbar #$e'// = 2*nhash #$e'//?

  Caso ind.2, **e = #e_1@e_2//**) **HI$_1$) ¡nbar e_1 = 2*nhash e_1!**
  HI$_2$) ¡nbar e_2 = 2*nhash e_2!
  TI) ¿nbar #e_1@e_2// = 2*nhash #e_1@e_2//?

  - Se plantean los casos

¡Observar cómo se aplica el principio de inducción estructural!

¡Observar las suposiciones!

Principio de inducción es

- Definición *inductiva* de XBE
  - **Regla base 1:** △ está en XBE
  - **Regla base 2:** ▽ está en XBE
  - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
  - **Regla inductiva 2:** si e_1 está en XBE y e_2 está en XBE, entonces $\#e_1@e_2//$ está en XBE

- **Prop.:** ¿nbar x = 2*nhash x

Dem.: sea **e** un elemento de **XBE**. Por ppio. de ind. en la estructura de **XBE**

Caso base 1, **e** = △) ¿nbar △ = 2*nhash △?

nbar △
= (nbar.1)
0

2*nhash △
= (nhash.1)
2*0
= (aritm.)
0

Vale este caso

Los casos base son iguales

Caso base 2, **e** = ▽) ¿nbar ▽ = 2*nhash ▽?

nbar ▽
= (nbar.2)
0

2*nhash ▽
= (nhash.2)
2*0
= (aritm.)
0

Vale este caso

Principio de inducción es

- Definición *inductiva* de XBE
  - **Regla base 1:** △ está en XBE
  - **Regla base 2:** ▽ está en XBE
  - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
  - **Regla inductiva 2:** si $\mathbf{e_1}$ está en XBE y $\mathbf{e_2}$ está en XBE, entonces $\mathbf{\#e_1@e_2//}$ está en XBE

- **Prop.:** ¿`nbar x = 2*nhash x`

Dem.: sea **e** un elemento de **XBE**. Por ppio. de ind. en la estructura de **XBE**

Caso ind.1, **e** = **#$e'//**)

HI) ¡`nbar e' = 2*nhash e'`!

TI) ¿`nbar #$e'// = 2*nhash #$e'//`?

`nbar #$e'//`	`2*nhash #$e'//`
= (nbar.3)	= (nhash.3)
`2 + nbar e'`	`2*(1 + nhash e')`
= (HI)	= (aritm.)
`2 + 2*nhash e'`	`2 + 2*nhash e'`

¡Observar el uso de la nueva suposición!

Vale este caso

Principio de inducción es

- Definición *inductiva* de XBE
  - **Regla base 1:** △ está en XBE
  - **Regla base 2:** ▽ está en XBE
  - **Regla inductiva 1:** si **e** está en XBE, entonces **#$e//** está en XBE
  - **Regla inductiva 2:** si e_1 está en XBE y e_2 está en XBE, entonces $\#e_1@e_2//$ está en XBE

- **Prop.:** ¿nbar x = 2*nhash x

Dem.: sea **e** un elemento de **XBE**. Por ppio. de ind. en la estructura de **XBE**

Caso ind.2, **e** = $\#e_1@e_2//$) HI$_1$) ¡nbar e_1 = 2*nhash e_1!

HI$_2$) ¡nbar e_2 = 2*nhash e_2!

TI) ¿nbar $\#e_1@e_2//$ = 2*nhash $\#e_1@e_2//$?

¡Observar el uso de las nuevas suposiciones!

nbar $\#e_1@e_2//$

= (nbar.4)

2 + nbar e_1 + nbar e_2

= (HI1)

2 + 2*nhash e_1 + nbar e_2

= (HI2)

2 + 2*nhash e_1 + 2*nhash e_2

2*nhash $\#e_1@e_2//$

= (nhash.2)

2*(1 + nhash e_1 + nhash e_2)

= (aritm.)

2 + 2*nhash e_1 + 2*nhash e_2

Vale este caso

Principio de inducción estructural

- ¿Cómo organizar una demostración por inducción?
  - Se decide sobre qué elemento usar inducción estructural
  - Se plantean ***todos*** los casos con sus hipótesis
  - Se demuestran los casos planteados
    (*en cualquier orden*)
    - En algunas propiedades, puede ser necesario hacer inducción en otro elemento *dentro* de la demostración de algún caso

Principio de inducción estructural

- ¿Por qué funciona el principio de inducción estructural?

Principio de inducción estructural

- ¿Por qué funciona el principio de inducción estructural?
  - ¿Por qué se puede suponer que la propiedad vale sobre las partes de un elemento inductivo?

Principio de inducción estructural

- ¿Por qué funciona el principio de inducción estructural?
  - ¿Por qué se puede suponer que la propiedad vale sobre las partes de un elemento inductivo?
    - ¡Porque no se puede descomponer infinitamente!
    - El orden **“es parte de”** es *bien fundado*

Principio de inducción estructural

- ¿Por qué funciona el principio de inducción estructural?
  - ¿Por qué se puede suponer que la propiedad vale sobre las partes de un elemento inductivo?
    - ¡Porque no se puede descomponer infinitamente!
    - El orden **“es parte de”** es *bien fundado*
      - El único punto de la demostración que se podría desconfiar tiene la misma *estructura* que toda la demostración
      - Como es punto no se puede encontrar infinitamente, es seguro suponer que en las partes vale como ya demostrado

Inducción matemática

Inducción/recursión estructural en números

- Definición *inductiva* de $\mathbb{N}$
  - **Regla base:** 0 está en $\mathbb{N}$
  - **Regla inductiva:** si **n** está en $\mathbb{N}$, entonces **n+1** está en $\mathbb{N}$
- ¿Cómo sería el principio de inducción estructural sobre $\mathbb{N}$?

Inducción/recursión estructural en números

- Definición *inductiva* de $\mathbb{N}$
  - **Regla base:** 0 está en $\mathbb{N}$
  - **Regla inductiva:** si **n** está en $\mathbb{N}$, entonces **n+1** está en $\mathbb{N}$
- ¿Cómo sería el principio de inducción estructural sobre $\mathbb{N}$?
  - ¿para todo $x \in \mathbb{N}. P(x)$? es equivalente a

    Caso base, **n = 0**) ¿**P(0)**?

    Caso inductivo, **n = n'+1**)

    HI) ¡**P(n')**!

    TI) ¿**P(n'+1)**?

Inducción/recursión estructural en números

- Definición *inductiva* de $\mathbb{N}$
  - **Regla base:** 0 está en $\mathbb{N}$
  - **Regla inductiva:** si **n** está en $\mathbb{N}$, entonces **n+1** está en $\mathbb{N}$
- ¿Cómo sería el principio de inducción estructural sobre $\mathbb{N}$?
  - ¿para todo $x \in \mathbb{N}. P(x)$? es equivalente a

    Caso base, **n = 0**) ¿**P(0)**?
    Caso inductivo, **n = n'+1**)
    HI) ¡**P(n')**!
    TI) ¿**P(n'+1)**?

Usualmente se escribe de esta otra forma

Caso base, **n = 0**) ¿**P(0)**?
Caso inductivo, **n = (n−1)+1**)
HI) ¡**P(n−1)**!
TI) ¿**P(n)**?

Inducción/recursión estructural en números

- Definición *inductiva* de $\mathbb{N}$
  - **Regla base:** 0 está en $\mathbb{N}$
  - **Regla inductiva:** si **n** está en $\mathbb{N}$, entonces **n+1** está en $\mathbb{N}$
- ¡El principio de inducción estructural sobre $\mathbb{N}$ es exactamente la ***inducción matemática*** usual!
  - ¿para todo $x \in \mathbb{N}.P(x)$? es equivalente a

    Caso base, **n = 0**) ¿**P(0)**?
    Caso inductivo, **n = n'+1**)
    HI) ¡**P(n')**!
    TI) ¿**P(n'+1)**?

Usualmente se escribe de esta otra forma

Caso base, **n = 0**) ¿**P(0)**?
Caso inductivo, **n = (n−1)+1**)
HI) ¡**P(n−1)**!
TI) ¿**P(n)**?

Inducción/recursión estructural en números

- **Prop.:** ¿para todo **x**. `fact x = factL x`?

  Dem.: Sea **n** un número natural. Por ppio. de ind. en la estructura de **n**

  Caso base, **n = 0**) ¿`fact 0 = factL 0`?

  ```
    fact 0                          factL 0
  =                  (fact.1)     =                  (factL.1)
    1                               1
  ```

  Vale este caso

  Caso ind., **n = n'+1**) **HI)** ¡`fact (n-1) = factL (n-1)`!

  TI) ¿`fact n = factL n`?

  ```
    fact n                          factL n
  =                  (fact.2 -- n=n) =                  (factL.2, n=n)
    n * fact (n-1)                  factL (n-1) * n
  =                  (aritm.)     =                  (HI)
    fact (n-1) * n                  fact (n-1) * n
  ```

  Vale este caso

  ¡La suposición que faltaba!

Inducción y recursión en Haskell

Inducción y recursión en Haskell

- ¿Cómo usar todas estas técnicas en Haskell?
  - ¿Definición de funciones por recursión estructural?

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn :: Mapa -> Cofre
tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

cavarYDar c     = c   -- Implementaciones triviales
caminarYDar n c = c   -- Podrían complicarse con un
girarYDar d c   = c   --   terreno y verificaciones…
```

Inducción y recursión en Haskell

- ¿Cómo usar todas estas técnicas en Haskell?
  - ¿Definición de funciones por recursión estructural?

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

comprimido :: Mapa -> Mapa -- Comprime tramos rectos consecutivos
comprimido (Equis c)    = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)   = Giro d (comprimido m)

compRecto n (Recto n' m) = Recto (n+n') m
compRecto n m            = Recto n m
```

Inducción y recursión en Haskell

- ¿Cómo usar todos estas técnicas en Haskell?
  - ¿Demostraciones usando el ppio. de ind. estr.?

Inducción y recu

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

comprimido (Equis c)   = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)  = Giro d (comprimido m)
```

- ¿Cómo usar todos
  - ¿Demostraciones usando el ppio. de ind. estr.?

Prop: ¿para todo **x**. `tesoroEn (comprimido x) = tesoroEn x`?

Dem:

Inducción y recu

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

comprimido (Equis c)   = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)  = Giro d (comprimido m)
```

- ¿Cómo usar todos
  - ¿Demostraciones usando el ppio. de ind. estr.?

Prop: ¿para todo **x**. `tesoroEn (comprimido x) = tesoroEn x`?

Dem: sea **m** un mapa. Por ind. en la estructura de **m**

Inducción y recu

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

comprimido (Equis c)   = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)  = Giro d (comprimido m)
```

- ¿Cómo usar todos
  - ¿Demostraciones usando el ppio. de ind. estr.?

Prop: ¿para todo **x**. `tesoroEn (comprimido x) = tesoroEn x`?

Dem: sea **m** un mapa. Por ind. en la estructura de **m**

Caso base, **m** = `Equis` **c**)

Caso ind.1, **m** = `Recto` **n m'**)

Caso ind.2, **m** = `Giro` **d m'**)

Inducción y recu

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

comprimido (Equis c)   = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)  = Giro d (comprimido m)
```

- ¿Cómo usar todos
  - ¿Demostraciones usando el ppio. de ind. estr.?

Prop: ¿para todo **x**. `tesoroEn (comprimido x) = tesoroEn x`?

Dem: sea **m** un mapa. Por ind. en la estructura de **m**

Caso base, **m** = `Equis c`) ¿`tesoroEn (comprimido (Equis c)) = tesoroEn (Equis c)`?

Caso ind.1, **m** = `Recto n m'`)

HI) ¡`tesoroEn (comprimido m') = tesoroEn m'`!

TI) ¿`tesoroEn (comprimido (Recto n m')) = tesoroEn (Recto n m')`?

Caso ind.2, **m** = `Giro d m'`)

HI) ¡`tesoroEn (comprimido m') = tesoroEn m'`!

TI) ¿`tesoroEn (comprimido (Giro d m')) = tesoroEn (Giro d m')`?

Inducción y recu

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

comprimido (Equis c)   = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)  = Giro d (comprimido m)
```

- ¿Cómo usar todos
  - ¿Demostraciones usando el ppio. de ind. estr.?

Prop: ¿para todo **x**. `tesoroEn (comprimido x) = tesoroEn x`?

Dem: sea **m** un mapa. Por ind. en la estructura de **m**

Caso base, **m** = `Equis c`) ¿`tesoroEn (comprimido (Equis c)) = tesoroEn (Equis c)`?

```
  tesoroEn (comprimido (Equis c))
=                                  (comprimido.1)
  tesoroEn (Equis c)
=                                  (tesoroEn.1)
  cavarYDar c
```

```
  tesoroEn (Equis c)
=                                  (tesoroEn.1)
  cavarYDar c
```

Vale este caso

Inducción y recu

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

comprimido (Equis c)   = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)  = Giro d (comprimido m)
```

- ¿Cómo usar todos
  - ¿Demostraciones usando el ppio. de ind. estr.?

Prop: ¿para todo **x**. `tesoroEn (comprimido x) = tesoroEn x`?

Dem: sea **m** un mapa. Por ind. en la estructura de **m**

Caso ind.2, **m** = `Giro d m'`) HI) ¡`tesoroEn (comprimido m') = tesoroEn m'`!

TI) ¿`tesoroEn (comprimido (Giro d m')) = tesoroEn (Giro d m')`?

```
  tesoroEn (comprimido (Giro d m'))
=                                   (comprimido.3)
  tesoroEn (Giro d (comprimido m'))
=                                   (tesoroEn.3)
  girarYDar d (tesoroEn (comprimido m'))
=                                   (HI)
  girarYDar d (tesoroEn m')
```

```
  tesoroEn (Giro d m')
=                                   (tesoroEn.3)
  girarYDar d (tesoroEn m')
```

Vale este caso

Inducción y recu

```
data Mapa = Equis Cofre | Recto Int Mapa | Giro Dir Mapa

tesoroEn (Equis c)   = cavarYDar c
tesoroEn (Recto n m) = caminarYDar n (tesoroEn m)
tesoroEn (Giro d m)  = girarYDar d (tesoroEn m)

comprimido (Equis c)   = Equis c
comprimido (Recto n m) = compRecto n (comprimido m)
comprimido (Giro d m)  = Giro d (comprimido m)
```

- ¿Cómo usar todos
  - ¿Demostraciones usando el ppio. de ind. estr.?

Prop: ¿para todo x. `tesoroEn (comprimido x) = tesoroEn x`?

Dem: sea **m** un mapa. Por ind. en la estructura de **m**

Caso ind.1, **m** = `Recto n m'`) **HI)** ¡`tesoroEn (comprimido m') = tesoroEn m'`!

TI) ¿`tesoroEn (comprimido (Recto n m')) = tesoroEn (Recto n m')`?

```
  tesoroEn (comprimido (Recto n m'))
=                                  (comprimido.2)        x
  tesoroEn (compRecto n (comprimido m'))
=                                  (Lema 1, x = comprimido m')
  caminarYDar n (tesoroEn (comprimido m'))
=                                  (HI)
  caminarYDar n (tesoroEn m')
```

```
  tesoroEn (Recto n m')
=                                  (tesoroEn.2)
  caminarYDar n (tesoroEn m')
```

Lema 1: ¿para todo x.
`tesoroEn (compRecto n x) = caminarYDar n (tesoroEn x)`?

Valen así todos los casos

Listas

Listas: motivación

- Se trabajaron un par de tipos similares

```
data MapaB = EquisB | RectoB Int MapaB | GiroB Dir MapaB
data Pizza = Prepizza | Capa Ingrediente Pizza
data Aleph = Z | S Aleph
```

Listas: motivación

- Se trabajaron un par de tipos similares

```
data MapaB = EquisB | RectoB Int MapaB | GiroB Dir MapaB
data Pizza = Prepizza | Capa Ingrediente Pizza
data Aleph = Z | S Aleph
```

  - ¿Se podrán generalizar estos tipos para quedarse solamente con la estructura, sin significado?

Listas: motivación

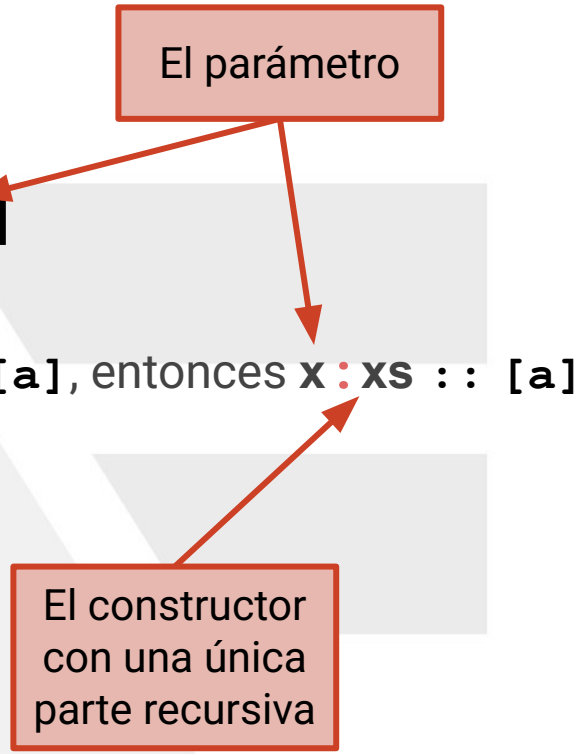
- Se trabajaron algunos tipos similares

```
data MapaB = EquisB | RectoB Int MapaB | GiroB Dir MapaB
data Pizza = Prepizza | Capa Ingrediente Pizza
data Aleph = Z | S Aleph
```

  - ¿Se podrán generalizar estos tipos para quedarse solamente con la estructura, sin significado?
    - Para generalizar deben usarse *parámetros*…
    - ¿Qué tienen en común?
      - Un constructor base
      - Uno (o más) constructor(es) con una *única* parte recursiva

Listas: definición

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si **`x`** `::` **`a`**, **`xs`** `::` `[a]`, entonces **`x`** `:` **`xs`** `::` `[a]`

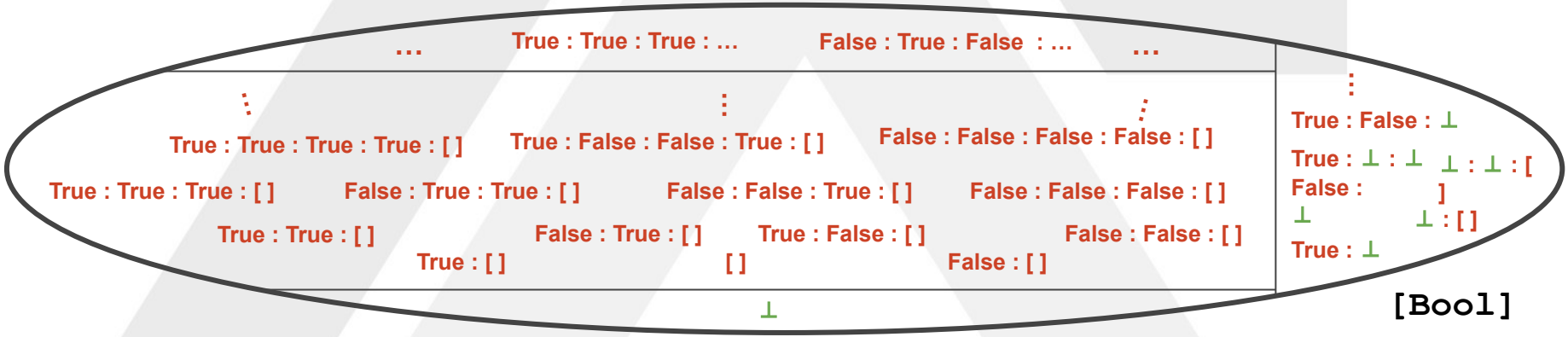


Listas: definición

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si **`x`** `::` **`a`**, **`xs`** `::` `[a]`, entonces **`x`** `:` **`xs`** `::` `[a]`
- ¿Qué conjunto es `[Bool]`? ¿Y `[Int]`?

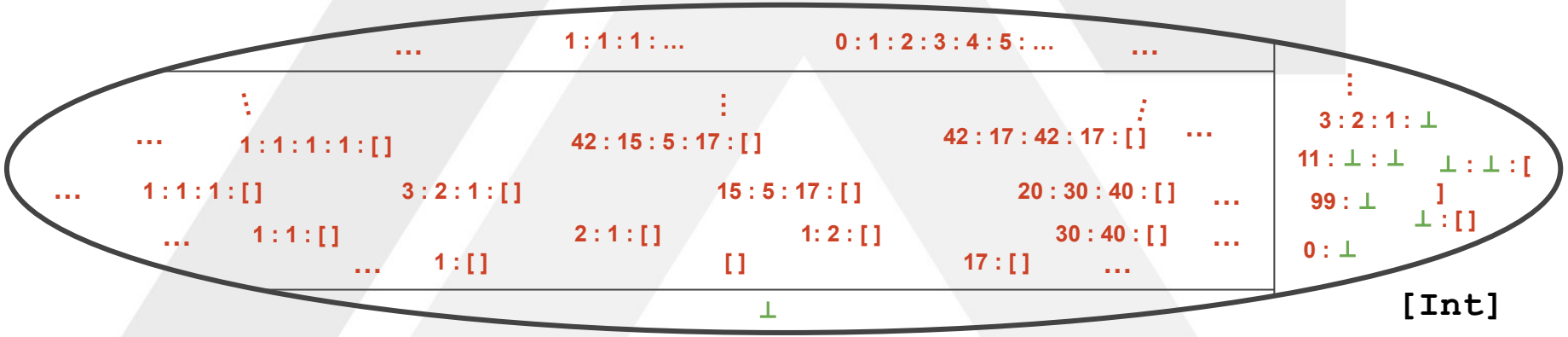
Listas: definición

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x : xs :: [a]`
- ¿Qué conjunto es `[Bool]`?



Listas: definición

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x : xs :: [a]`
- ¿Qué conjunto es `[Int]`?



Listas: notación

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[] :: [a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x:xs :: [a]`
- Se puede usar notación especial
  - `[10,20,30]` en lugar de `10:20:30:[]`
  - `"abc"` en lugar de `'a':'b':'c':[]` (o `['a','b','c']`)
  - También hay otras que no veremos
    - secuencias aritméticas y listas por comprensión
  - Distintas notaciones se pueden combinar `10:[20,30]`

Listas: motivación

- ¿Cómo expresar los ejemplos como instancias de `[a]`?

```
type MapaL   = [ MapaNR ]
data MapaNR  = RectoNR Int | GiroNR Dir
```

  - ¿Se podrán transformar mapas de **MapaB** en **MapaL**?
    - Por recursión en la estructura de **MapaB**

```
mb2ml :: MapaB -> MapaL
mb2ml EquisB        = []
mb2ml (RectoB n m) = RectoNR n : mb2ml m
mb2ml (GiroB d m)   = GiroNR d : mb2ml m
```

  - ¿Y viceversa?
    - Hace falta poder definir funciones sobre listas…

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[] :: [a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x : xs :: [a]`
- Funciones por recursión estructural sobre listas

```
f :: [A] -> B
f []     = ...
f (x:xs) = ... x ... f xs ...
```

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x:xs :: [a]`
- Funciones por recursión estructural sobre listas

```
f :: [A] -> B
f []     = ...
f (x:xs) = ... x ... f xs ...
```

Prestar atención:
¡son metavariables!

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si **`x`** `::` **`a`**, **`xs`** `::` `[a]`, entonces **`x`** `:` **`xs`** `::` `[a]`
- Funciones por recursión estructural sobre listas

```
len :: [a] -> Int                    -- Cantidad de elementos
```

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si **`x`** `::` **`a`**, **`xs`** `::` `[a]`, entonces **`x`** `:` **`xs`** `::` `[a]`
- Funciones por recursión estructural sobre listas

```
len :: [a] -> Int                    -- Cantidad de elementos
```

Se decide usar recursión estructural

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x:xs :: [a]`
- Funciones por recursión estructural sobre listas

```
len :: [a] -> Int                -- Cantidad de elementos
len []     = ...
len (x:xs) = ... x ... len xs ...
```

Se plantea la estructura en base a las reglas

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[] :: [a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x:xs :: [a]`
- Funciones por recursión estructural sobre listas

```
len :: [a] -> Int                -- Cantidad de elementos
len []     = ...
len (x:xs) = 1 + len xs
```

Se definen los casos inductivos

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[] :: [a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x:xs :: [a]`
- Funciones por recursión estructural sobre listas

```
len :: [a] -> Int                -- Cantidad de elementos
len []     = 0
len (x:xs) = 1 + len xs
```

Se completa con los casos base

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si **`x`** `::` **`a`**, **`xs`** `::` `[a]`, entonces **`x`** `:` **`xs`** `::` `[a]`
- Funciones por recursión estructural sobre listas

```
append :: [a] -> [a] -> [a] -- Agregar una lista
```

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si **`x`** `::` **`a`**, **`xs`** `::` `[a]`, entonces **`x`** `:` **`xs`** `::` `[a]`
- Funciones por recursión estructural sobre listas

```
append :: [a] -> [a] -> [a] -- Agregar una lista
```

Se decide usar recursión estructural

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x : xs :: [a]`
- Funciones por recursión estructural sobre listas

```
append :: [a] -> [a] -> [a] -- Agregar una lista
append []     ys = ...
append (x:xs) ys = ... x ... append xs ys ...
```

Se plantea la estructura en base a las reglas

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x : xs :: [a]`
- Funciones por recursión estructural sobre listas

```
append :: [a] -> [a] -> [a] -- Agregar una lista
append []     ys = ...
append (x:xs) ys = x : append xs ys
```

Se definen los casos inductivos

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[] :: [a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x:xs :: [a]`
- Funciones por recursión estructural sobre listas

```
append :: [a] -> [a] -> [a] -- Agregar una lista
append []     ys = ys
append (x:xs) ys = x : append xs ys
```

Se completa con los casos base

Recursión estructural sobre listas

- Definición *inductiva* de listas `[a]`
  - **Regla base:** `[]` `::` `[a]`
  - **Regla inductiva:** si `x :: a`, `xs :: [a]`, entonces `x:xs :: [a]`
- Funciones por recursión estructural sobre listas

```
append :: [a] -> [a] -> [a] -- Agregar una lista
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys    -- Una abreviatura cómoda
```

Recursión estructural sobre listas

- Funciones sobre listas `[a]`
  - Ejemplos

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

```
10:20:30:40:[]
10 : [20,30,40]
10:20:[] ++ [30,40]
append [10,20] [30,40]
```

```
len [10,20,30,40] = ??
len (append xs ys) = ??
len (xs ++ ys) = ??
```

Recursión estructural sobre listas

- Funciones sobre listas `[a]`
  - Ejemplos

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

```
10:20:30:40:[]
10 : [20,30,40]
10:20:[] ++ [30,40]
append [10,20] [30,40]
```

```
len [10,20,30,40] = 4
```

```
len (append xs ys) = ??
len (xs ++ ys) = ??
```

Recursión estructural sobre listas

- Funciones sobre listas `[a]`
  - Ejemplos

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

```
10:20:30:40:[]
10 : [20,30,40]
10:20:[] ++ [30,40]
append [10,20] [30,40]
```

```
len [10,20,30,40] = 4
```

```
len (append xs ys) = len xs + len ys
len (xs ++ ys) = ??
```

Recursión estructural sobre listas

- Funciones sobre listas `[a]`
  - Ejemplos

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

```
10:20:30:40:[]
10 : [20,30,40]
10:20:[] ++ [30,40]
append [10,20] [30,40]
```

```
len [10,20,30,40] = 4
```

```
len (append xs ys) = len xs + len ys
len (xs ++ ys) = len xs + len ys
```

Demostraciones con inducci

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

- Funciones sobre listas **`[a]`**
  - **Prop:** ¿para todo **xs** e **ys**.

    `len (xs ++ ys) = len xs + len ys?`

Demostraciones con inducci

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

- Funciones sobre listas **`[a]`**
  - **Prop:** ¿para todo **xs** e **ys**.

    `len (xs ++ ys) = len xs + len ys?`
  - ¿Qué hace falta para demostrar esto?

Demostraciones con inducci

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

- Funciones sobre listas **`[a]`**
  - **Prop:** ¿para todo **xs** e **ys**.

    `len (xs ++ ys) = len xs + len ys?`
  - ¿Qué hace falta para demostrar esto?
    - Elegir dos elementos cualesquiera y usar...
    - ¡el principio de inducción estructural sobre listas!

Demostraciones con inducci

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

- Funciones sobre listas **`[a]`**
  - **Prop:** ¿para todo **xs** e **ys**.

    `len (xs ++ ys) = len xs + len ys?`
  - ¿Qué hace falta para demostrar esto?
    - Elegir dos elementos cualesquiera y usar...
    - ¡el principio de inducción estructural sobre listas!
      - ¿para todo **xs**. P(**xs**)? es equivalente a
        - Caso base) ¿P(`[]`)?
        - Caso ind.) **HI)** ¡P(**xs'**)!

          TI) ¿P(**x**`:`**xs'**)?

¡Solamente vale para la parte finita y totalmente definida (inductiva) de **`[a]`**!

Demostraciones con inducci

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

- Funciones sobre listas **[a]**
  - **Prop:** ¿para todo **xs** e **ys**.
    len (xs ++ ys) = len xs + len ys?
  - **Dem:** Sean **zs** y **ws** listas cualesquiera (finitas y totalmente definidas).
    Por ppio. de ind. en la estructura de **zs** es equivalente demostrar

    Caso base, **zs = []**)

    ¿**len ([] ++ ws) = len [] + len ws?**

    Caso ind., **zs = x:xs'**)

    HI) ¡**len (xs' ++ ws) = len xs' + len ws!**

    TI) ¿**len ((x:xs') ++ ws) = len (x:xs') + len ws?**

Demostraciones con inducci

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

- Funciones sobre listas `[a]`
  - **Prop:** ¿para todo **xs** e **ys**.

    `len (xs ++ ys) = len xs + len ys?`
  - **Dem:** Caso base, **zs** = `[]`) ¿`len ([] ++ ws) = len [] + len ws?`

```
  len ([] ++ ws)
=                    ((++) y append.1)
  len ws
```

```
  len [] + len ws
=                    (len.1)
  0 + len ws
=                    (aritm.)
  len ws
```

Vale este caso

Demostraciones con inducci

```
len :: [a] -> Int
len []     = 0
len (x:xs) = 1 + len xs

append :: [a] -> [a] -> [a]
append []     ys = ys
append (x:xs) ys = x : append xs ys

xs ++ ys = append xs ys
```

- Funciones sobre listas `[a]`
  - **Prop:** ¿para todo **xs** e **ys**.

    `len (xs ++ ys) = len xs + len ys?`
  - **Dem:** Caso ind., **zs = x:xs')** **HI)** ¡`len (xs' ++ ws) = len xs' + len ws!`

    TI) ¿`len ((x:xs')++ ws) = len (x:xs') + len ws?`

```
  len ((x:xs') ++ ws)
=                    ((++) y append.2)
  len (x : (xs'++ws))
=                    (len.2)
  1 + len (xs'++ws)
=                    (HI)
  1 + (len xs' + len ws)
```

```
  len (x:xs') + len ws
=                    (len.2)
  (1 + len xs') + len ws
=                    (aritm.)
  1 + (len xs' + len ws)
```

Vale este caso

Vale la propiedad

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - No recursivas

```
head :: [a] -> a          -- El 1er elemento
head ...

tail :: [a] -> [a]        -- Todos menos el 1er elemento
tail ...

null :: [a] -> Bool       -- Indica si la lista es vacía
null ...
```

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - No recursivas

```
head :: [a] -> a          -- El 1er elemento
head (x:xs) = x           -- Parcial en []

tail :: [a] -> [a]        -- Todos menos el 1er elemento
tail (x:xs) = xs          -- Parcial en []

null :: [a] -> Bool       -- Indica si la lista es vacía
null [] = True
null _  = False
```

Recursión estructural sobre listas

- Funciones sobre listas `[a]`
  - ¿Cuáles son los pasos adecuados para armar funciones recursivas estructurales?

Recursión estructural sobre listas

- Funciones sobre listas `[a]`
  - ¿Cuáles son los pasos adecuados para armar funciones recursivas estructurales?
    - Plantear el tipo de la función y probar ejemplos
    - Decidir usar recursión estructural (y *sobre qué elemento*)
    - *Plantear* el esquema de recursión estructural
    - Definir los casos inductivos
    - Completar con los casos base
    - Hacer ajustes de ser necesario

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
```

- Funciones sobre listas **`[a]`**
  - Recursivas estructurales

```
sum  :: [Int] -> Int    -- La suma de todos los números
sum ...

prod :: [Int] -> Int    -- El producto de todos los números
prod ...

and  :: [Bool] -> Bool  -- La conjunción de
and ...                 --  todos los booleanos
```

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
sum  :: [Int] -> Int     -- La suma de todos los números
sum ...
```

`sum [10,20,30] = 10+20+30 = 60`

```
prod :: [Int] -> Int     -- El producto de todos los números
prod ...
```

`prod [10,20,30] = 10*20*30 = 6000`

```
and  :: [Bool] -> Bool   -- La conjunción de
and ...                  --  todos los booleanos
```

`and [True,False,True] = True&&False&&True = False`

Se prueban ejemplos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
sum  :: [Int] -> Int    -- La suma de todos los números
sum ...

prod :: [Int] -> Int    -- El producto de todos los números
prod ...

and  :: [Bool] -> Bool  -- La conjunción de
and ...                 --  todos los booleanos
```

Se decide hacer recursión estructural

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
```

- Funciones sobre listas `[a]`
  - Recursivas estructurales

```
sum  :: [Int] -> Int    -- La suma de todos los números
sum []       = ...
sum (n:ns)   = ... n ... sum ns ...

prod :: [Int] -> Int    -- El producto de todos los números
prod []      = ...
prod (n:ns)  = ... n ... prod ns ...

and  :: [Bool] -> Bool  -- La conjunción de
and []       = ...      --  todos los booleanos
and (b:bs)   = ... b ... and bs ...
```

Se plantea la estructura en base a las reglas

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
```

- Funciones sobre listas `[a]`
  - Recursivas estructurales

```
sum  :: [Int] -> Int    -- La suma de todos los números
sum []       = ...
sum (n:ns)   = n + sum ns

prod :: [Int] -> Int    -- El producto de todos los números
prod []      = ...
prod (n:ns) = n * prod ns

and  :: [Bool] -> Bool  -- La conjunción de
and []       = ...      --  todos los booleanos
and (b:bs)   = b && and bs
```

Se definen los casos inductivos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
```

- Funciones sobre listas `[a]`
  - Recursivas estructurales

```
sum  :: [Int] -> Int    -- La suma de todos los números
sum []       = 0
sum (n:ns)   = n + sum ns

prod :: [Int] -> Int    -- El producto de todos los números
prod []      = 1
prod (n:ns) = n * prod ns

and  :: [Bool] -> Bool  -- La conjunción de
and []       = True     --  todos los booleanos
and (b:bs)   = b && and bs
```

Se completa con los casos base

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **`[a]`**
  - Recursivas estructurales

```
sum  :: [Int]  -> Int  -- La suma de todos los números
prod :: [Int]  -> Int  -- El producto de todos los números
and  :: [Bool] -> Bool -- La conjunción de todos los booleanos
```

  - ¿Cómo se eligen los casos base en funciones de listas?

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **`[a]`**
  - Recursivas estructurales

```
sum  :: [Int]  -> Int   -- La suma de todos los números
prod :: [Int]  -> Int   -- El producto de todos los números
and  :: [Bool] -> Bool  -- La conjunción de todos los booleanos
```

  - ¿Cómo se eligen los casos base en funciones de listas?
    - ¡Son el **neutro** de la operación usada en el caso inductivo!
    - Por eso se eligen al último…

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
uppers :: [Char] -> [Char] -- Pasa todas a mayúsculas
uppers ...

noVacias:: [[a]] -> [[a]]  -- Elimina elementos []
noVacias ...
```

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
uppers :: [Char] -> [Char] -- Pasa todas a mayúsculas
uppers ...
```

```
uppers "abc123" = "ABC123"
```

```
noVacias:: [[a]] -> [[a]]  -- Elimina elementos []
noVacias ...
```

```
noVacias [[1,2,3],[],[4,5]] = [[1,2,3],[4,5]]
```

Se prueban ejemplos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
uppers :: [Char] -> [Char] -- Pasa todas a mayúsculas
uppers ...

noVacias:: [[a]] -> [[a]]  -- Elimina elementos []
noVacias ...
```

Se decide usar recursión estructural

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
uppers :: [Char] -> [Char] -- Pasa todas a mayúsculas
uppers []          = ...
uppers (c:cs)      = ... c ... uppers cs ...

noVacias:: [[a]] -> [[a]]  -- Elimina elementos []
noVacias []        = ...
noVacias (xs:xss) = ... xs ... noVacias xss ...
```

Se plantea la estructura en base a las reglas

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
uppers :: [Char] -> [Char] -- Pasa todas a mayúsculas
uppers []          = ...
uppers (c:cs)      = upper c : uppers cs

noVacias:: [[a]] -> [[a]]  -- Elimina elementos []
noVacias []        = ...
noVacias (xs:xss) = if null xs
                     then noVacias xss
                     else xs : noVacias xss
```

Se definen los casos inductivos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
```

- Funciones sobre listas **[a]**
  - Recursivas estructurales

```
uppers :: [Char] -> [Char] -- Pasa todas a mayúsculas
uppers []          = []
uppers (c:cs)      = upper c : uppers cs

noVacias:: [[a]] -> [[a]]  -- Elimina elementos []
noVacias []        = []
noVacias (xs:xss) = if null xs
                    then noVacias xss
                    else xs : noVacias xss
```

Se completa con los casos base

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - La recursión estructural sirve para muchas funciones
    - Recordar que todas ellas terminan por construcción
  - ¿Habrá funciones recursivas que no sean estructurales?
    - ¿Qué problemas pueden surgir?
  - ¿Habrá funciones que no se pueden hacer por recursión?
    - ¿Por qué?
  - ¿Siempre es necesario usar recursión?
    - ¿Qué alternativas hay?

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a        -- El máximo de los elementos
maximum …
```

```
last :: [a] -> a           -- El último de los elementos
last …
```

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a        -- El máximo de los elementos
maximum …
```

```
maximum [42,99,13,17] = 99
```

```
last :: [a] -> a           -- El último de los elementos
last …
```

```
last [1,2,3] = 3
```

Se prueban ejemplos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a        -- El máximo de los elementos
maximum …
```

```
last :: [a] -> a           -- El último de los elementos
last …
```

Se decide usar recursión estructural

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = ...
maximum (x:xs) = ... x ... maximum xs ...
```

```
last :: [a] -> a         -- El último de los elementos
last []     = ...
last (x:xs) = ... x ... last xs ...
```

Se plantea la estructura en base a las reglas

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = ...
maximum (x:xs) = x `max` maximum xs


last :: [a] -> a         -- El último de los elementos
last []     = ...
last (x:xs) = last xs
```

Se definen los casos inductivos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = ...
maximum (x:xs) = x `max` maximum xs


last :: [a] -> a         -- El último de los elementos
last []     = ...
last (x:xs) = last xs
```

¡Son operaciones sin neutro!

Se completa con los casos base

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = error "No hay elementos"
maximum (x:xs) = x `max` maximum xs


last :: [a] -> a         -- El último de los elementos
last []     = error "No hay elementos"
last (x:xs) = last xs
```

Quedan funciones parciales…

Se completa con los casos base

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = error "No hay elementos"
maximum (x:xs) = x `max` maximum xs
```

```
last :: [a] -> a         -- El último de los elementos
last []     = error "No hay elementos"
last (x:xs) = last xs
```

… que no deberían usarse sin controlar

Se hacen ajustes si hace falta

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = error "No hay elementos"
maximum (x:xs) = if null xs
                 then …
                 else x `max` maximum xs

last :: [a] -> a         -- El último de los elementos
last []     = error "No hay elementos"
last (x:xs) = if null xs
              then …
              else last xs
```

… que no deberían usarse sin controlar

Se hacen ajustes si hace falta

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = error "No hay elementos"
maximum (x:xs) = if null xs
                 then x
                 else x `max` maximum xs

last :: [a] -> a         -- El último de los elementos
last []     = error "No hay elementos"
last (x:xs) = if null xs
              then x
              else last xs
```

… que no deberían usarse sin controlar

Se hacen ajustes si hace falta

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = error "No hay elementos"
maximum (x:xs) = if null xs
                 then x
                 else x `max` maximum xs

last :: [a] -> a         -- El último de los elementos
last []     = error "No hay elementos"
last (x:xs) = if null xs
              then x
              else last xs
```

¡Estos usos NO son estructurales!

Se hacen ajustes si hace falta

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum []     = error "No hay elementos"
maximum (x:xs) = if null xs
                 then x
                 else x `max` maximum xs

last :: [a] -> a         -- El último de los elementos
last []     = error "No hay elementos"
last (x:xs) = if null xs
              then x
              else last xs
```

Esto casos podrían omitirse…

Se hacen ajustes si hace falta

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos

maximum (x:xs) = if null xs
                 then x
                 else x `max` maximum xs

last :: [a] -> a         -- El último de los elementos

last (x:xs)  = if null xs
               then x
               else last xs
```

Esto casos podrían omitirse…

Se hacen ajustes si hace falta

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a        -- El máximo de los elementos
maximum (x:[]) = x
maximum (x:xs) = x `max` maximum xs


last :: [a] -> a           -- El último de los elementos
last (x:[])  = x
last (x:xs)  = last xs
```

… y controlarse de forma separada

Se hacen ajustes si hace falta

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
maximum :: [a] -> a      -- El máximo de los elementos
maximum (x:[]) = x
maximum (x:xs) = x `max` maximum xs


last :: [a] -> a         -- El último de los elementos
last (x:[])  = x
last (x:xs)  = last xs
```

Es más difícil apreciar la estructura

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert …
```

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert …
```

```
insert 42 [12,17,99] = [12,17,42,99]
```

Se analizan ejemplos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert …
```

Se decide usar recursión estructural

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert x []     = …
insert x (y:ys) = … x … y … insert x ys …
```

Se plantea la estructura en base a las reglas

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert x []     = …
insert x (y:ys) = if x <= y
                  then x : y : ys
                  else y : insert x ys
```

Se definen los casos inductivos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert x []     = [x]
insert x (y:ys) = if x <= y
                  then x : y : ys
                  else y : insert x ys
```

Se completa con los casos base

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert x []     = [x]
insert x (y:ys) = if x <= y
                  then x : y : ys
                  else y : insert x ys
```

¡Este uso NO es estructural!

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas…

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert x []     = [x]
insert x (y:ys) = if x <= y
                  then x : y : ys
                  else y : insert x ys
```

¡Este uso NO es estructural!

  - Llamaremos a esta forma ***recursión primitiva***
    - Es una forma similar a la estructural, con más permisos

Más funciones sobre listas

```
len :: [a] -> Int
append, (++) :: [a] -> [a] -> [a]
sum, prod :: [Int] -> Int
uppers :: [Char] -> [Char]
noVacias :: [[a]] -> [[a]]
maximum, last :: [a] -> a
```

- Funciones sobre listas **[a]**
  - Recursivas primitivas

```
insert :: a -> [a] -> [a]  -- El elemento agregado en orden
insert x []     = [x]
insert x (y:ys) = if x <= y
                  then x : y : ys
                  else y : insert x ys
```

¡Este uso NO es estructural!

  - Llamaremos a esta forma ***recursión primitiva***
    - Es una forma similar a la estructural, con más permisos

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Habrá funciones recursivas que no sean estructurales?
    - Sí, hay funciones **recursivas primitivas**
      - También terminan por construcción

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Habrá funciones recursivas que no sean estructurales?
    - Sí, hay funciones **recursivas primitivas**
  - ¿Toda función se puede hacer por recursión?

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Toda función se puede hacer por recursión?

```
promedio :: [Float] -> Float  -- El promedio de los números
```

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Toda función se puede hacer por recursión?

```
promedio :: [Float] -> Float  -- El promedio de los números
promedio (x:[]) = ...
promedio (x:xs) = ... x ... promedio xs ...
```

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Toda función se puede hacer por recursión?

```
promedio :: [Float] -> Float   -- El promedio de los números
promedio (x:[]) = ...
promedio (x:xs) = ... x ... promedio xs ...
```

¿Cómo incorporar x al promedio anterior? ¡La longitud de xs se perdió!

```
promedio [9,8] = 8.50
```

```
promedio [9,8] = ... 9 ... 8 ...
```

```
promedio [9,8,8,8] = 8.25
```

```
promedio [9,8,8,8] = ... 9 ... 8 ...
```

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Toda función se puede hacer por recursión?

```
promedio :: [Float] -> Float   -- El promedio de los números
promedio (x:[]) = ...
promedio (x:xs) = ... x ... promedio xs ...

promedio xs = sum xs / length xs
```

¿Cómo incorporar x al promedio anterior? ¡La longitud de xs se perdió!

¡Es mejor usar subtareas!

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Habrá funciones recursivas que no sean estructurales?
    - Sí, hay funciones **recursivas primitivas**
  - ¿Toda función se puede hacer por recursión?
    - Hay casos en que es mejor usar subtareas

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Habrá funciones recursivas que no sean estructurales?
    - Sí, hay funciones **recursivas primitivas**
  - ¿Toda función se puede hacer por recursión?
    - Hay casos en que es mejor usar subtareas
  - ¿Es mejor por recursión o por subtareas?

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Es mejor por recursión o por subtareas?

```
sumSqr :: [Int] -> Int      -- La suma de cuadrados
```

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Es mejor por recursión o por subtareas?

```
sumSqr :: [Int] -> Int      -- La suma de cuadrados
sumSqr []     = 0
sumSqr (n:ns) = n^2 + sumSqr ns
```

vs.

```
sumSqr ns = sum (sqrs ns)  -- Los cuadrados de cada número
    where sqrs []     = []
          sqrs (n:ns) = n^2 : sqrs ns
```

Más funciones sobre listas

- Funciones sobre listas `[a]`
  - ¿Habrá funciones recursivas que no sean estructurales?
    - Sí, hay funciones **recursivas primitivas**
    - Ej: `maximum`
  - ¿Toda función se puede hacer por recursión?
    - Hay casos en que es mejor usar subtareas
    - Ej: `promedio`
  - ¿Es mejor por recursión o por subtareas?
    - Hay casos donde da lo mismo cualquiera de ambas
    - Ej: `sumSqr`

Representaciones de datos

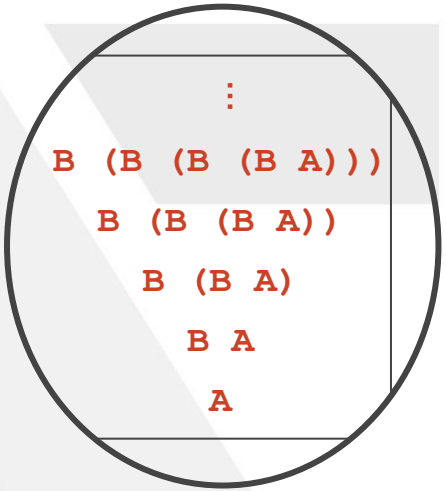
Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?

Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Qué representan los siguientes tipos de datos?

```
data TA = A | B TA
```

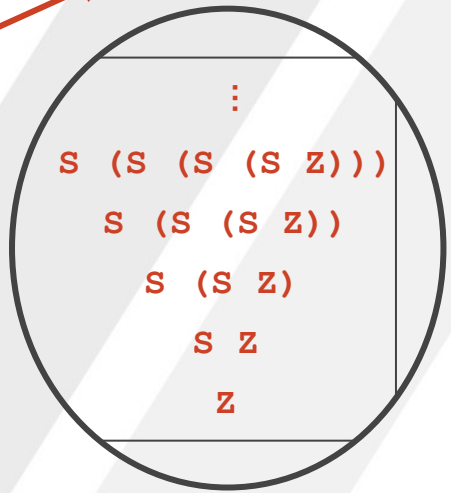


Representaciones de datos

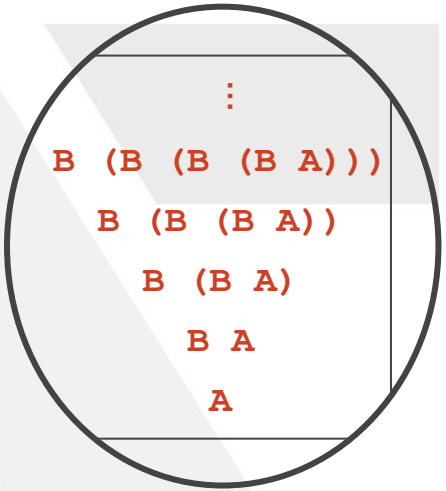
- ¿Qué cosas interesantes se pueden representar?
  - ¿Qué representan los siguientes tipos de datos?

```
data N = Z | S N
```

Cambio de nombre de **Aleph** a **N**...



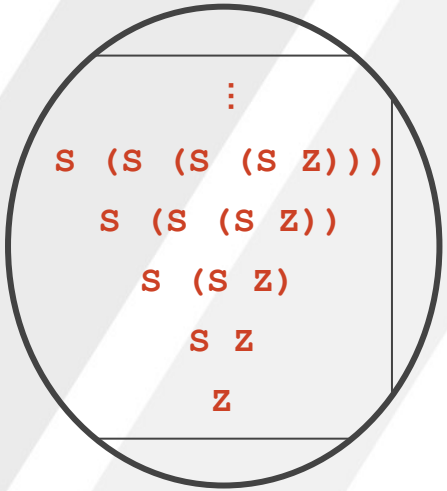

```
data TA = A | B TA
```



Representaciones de datos

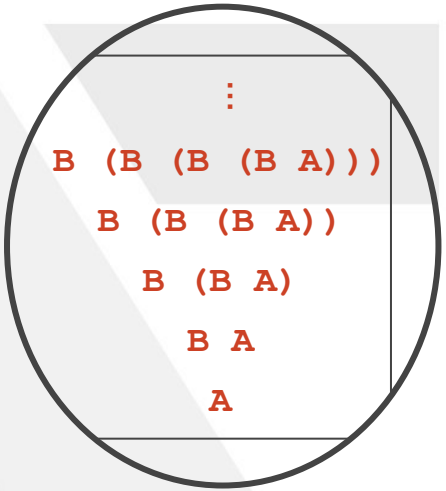
- ¿Qué cosas interesantes se pueden representar?
  - ¿Qué representan los siguientes tipos de datos?

```
data N = Z | S N
```



Nada (solamente estructura)

```
data TA = A | B TA
```



Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Cómo se relacionan **N** y **TA**?
    - **N** y **A** tienen la misma estructura

```
data N = Z | S N
n2ta :: N -> TA
n2ta Z     = A
n2ta (S n) = B (n2ta n)
```

```
data TA = A | B TA
ta2n :: TA -> N
ta2n A     = Z
ta2n (B a) = S (ta2n a)
```

  - Se puede demostrar que
    - `n2ta . ta2n = id :: TA->TA`

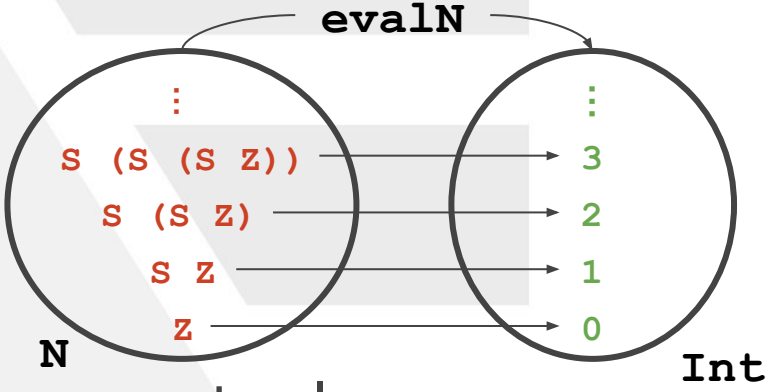
      y `ta2n . n2ta = id :: N->N`

Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Se puede hacer que representen algo?

```
data N = Z | S N

evalN :: N -> Int
evalN Z     = 0
evalN (S n) = 1 + evalN n
```



  - ¡**evalN** da significado a **N**!
  - **N** puede representar a los números naturales
    - Y usando **evalTA = evalN . ta2n**, **TA** también

Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Ya vimos esta idea?

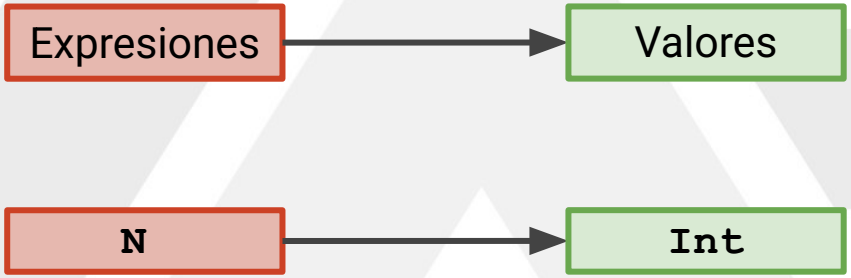
Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Ya vimos esta idea?
    - ¡Claro! En la primera clase…



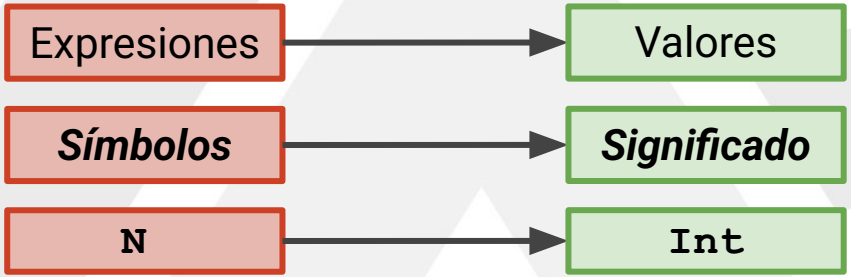
Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Ya vimos esta idea?
    - ¡Claro! En la primera clase…



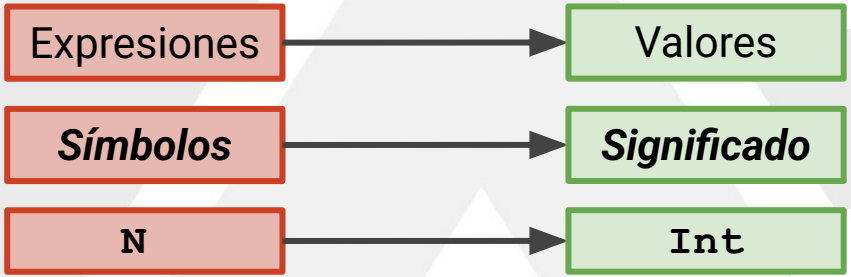
Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Ya vimos esta idea?
    - ¡Claro! En la primera clase…



Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Ya vimos esta idea?
    - ¡Claro! En la primera clase…



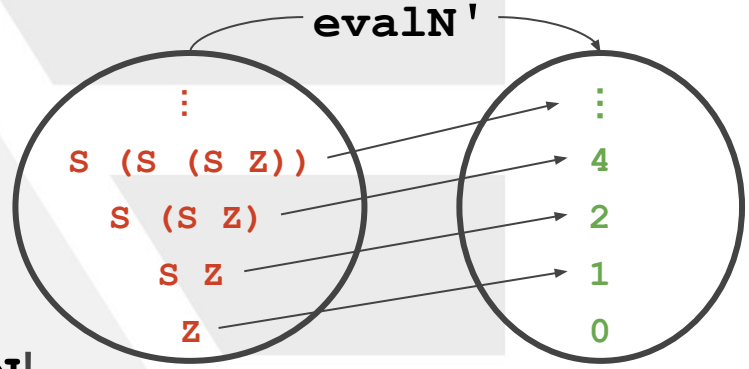
  - ¡Podemos usar el lenguaje de programación e inducción estructural para dar significados a símbolos!

Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿El significado es único?

```
data N = Z | S N

evalN' :: N -> Int
evalN' Z     = 1
evalN' (S n) = 2 * evalN' n
```



  - ¡**evalN'** da ***otro*** significado a **N**!
  - **N** puede representar a las potencias de 2
    - Y usando **evalTA' = evalN' . ta2n**, **TA** también



Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Y cómo sería la manipulación de símbolos?



Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Y cómo sería la manipulación de símbolos?
    - Una función que solamente trabaje con símbolos…



Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Y cómo sería la manipulación de símbolos?
    - Una función que solamente trabaje con símbolos…

```
data N = Z | S N

addN :: N -> N -> N
addN ...
```

```
addN (S(S(S Z))) (S(S Z))
     = S(S(S(S(S Z))))
```



Representaciones de datos

- ¿Qué cosas interesantes se pueden representar?
  - ¿Y cómo sería la manipulación de símbolos?
    - Una función que solamente trabaje con símbolos…

```
data N = Z | S N

addN :: N -> N -> N
addN Z     m = ...
addN (S n) m = ... addN n ... m ...
```

```
addN (S(S(S Z))) (S(S Z))
     = S(S(S(S(S Z))))
                  m
                m
```



Representaciones de datos

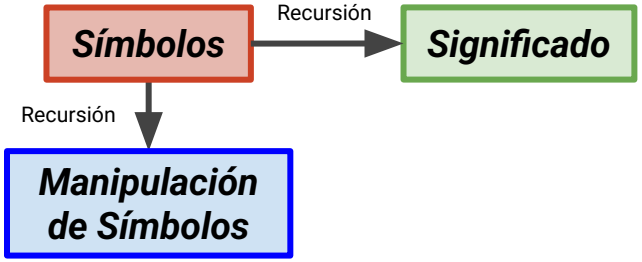
- ¿Qué cosas interesantes se pueden representar?
  - ¿Y cómo sería la manipulación de símbolos?
    - Una función que solamente trabaje con símbolos…

```
data N = Z | S N

addN :: N -> N -> N
addN Z     m = m
addN (S n) m = S (addN n m)
```

```
addN (S(S(S Z))) (S(S Z))
     = S(S(S(S(S Z))))
```

Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Y cómo sería la manipulación de símbolos?
    - Una función que solamente trabaje con símbolos…

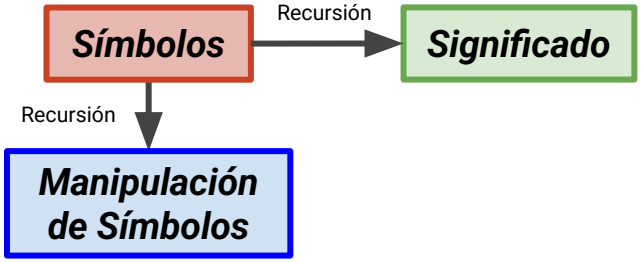
```
data N = Z | S N

addN :: N -> N -> N
addN Z     m = m
addN (S n) m = S (addN n m)
```

```
addN (S(S(S Z))) (S(S Z))
     = S(S(S(S(S Z)))) m
                       m
```

    - El tratamiento es puramente simbólico (sin significado)
      - Lo puede hacer un mecanismo automático…

Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Y cómo sería la manipulación de símbolos?
    - Otra función que solamente trabaje con símbolos…

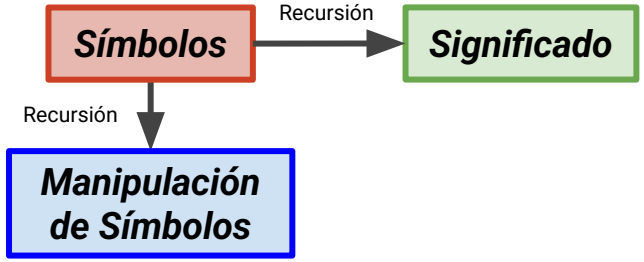
```
data N = Z | S N

prodN :: N -> N -> N
prodN Z     m = m
prodN (S n) m = addN (prodN n m) m
```

```
prodN (S(S(S Z))) (S(S Z))
    = S(S(S(S(S Z))))
```

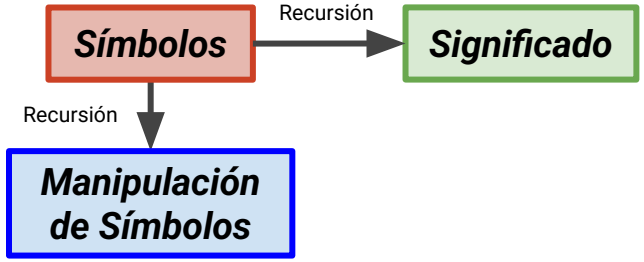
    - Nuevamente, el tratamiento es puramente simbólico

Representaciones de datos



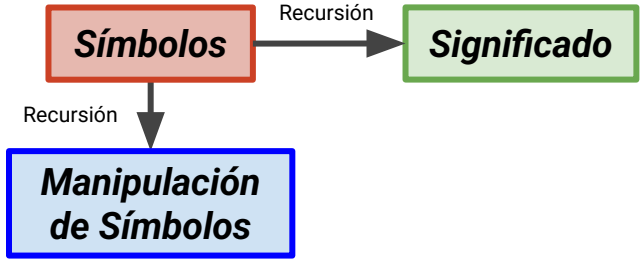
- ¿Qué cosas interesantes se pueden representar?
  - ¿Relación entre manipulación y significado?

Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Relación entre manipulación y significado?
    - Una propiedad que vincule las funciones…

Representaciones de datos



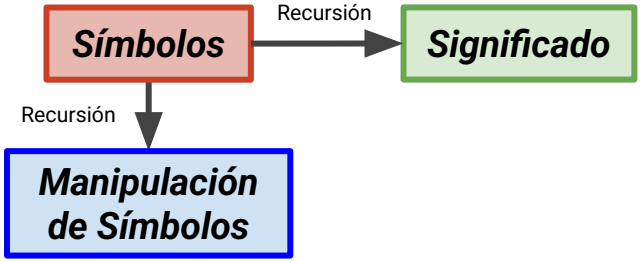
- ¿Qué cosas interesantes se pueden representar?
  - ¿Relación entre manipulación y significado?
    - Una propiedad que vincule las funciones…

      Prop: ¿para todo **n**. para todo **m**.

      ```
      evalN (addN n m) = evalN n + evalN m?
      ```

      Dem:

Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Relación entre manipulación y significado?
    - Una propiedad que vincule las funciones…

      Prop: ¿para todo **n**. para todo **m**.

      $$\texttt{evalN (addN n m)} = \texttt{evalN n} + \texttt{evalN m}?$$

      Dem: Sean n_0 y m_0 elementos totalmente definidos de **N**.

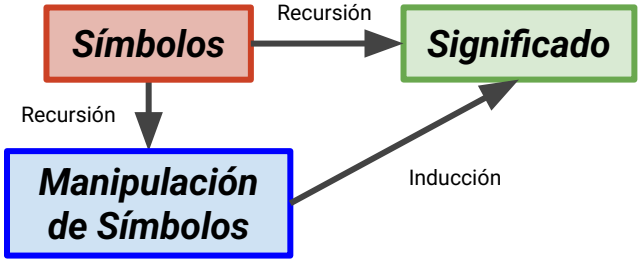
      Por ppio. de ind. en la estructura de n_0, se verá que

      Caso base n_0 = Z) ¿$\texttt{evalN (addN Z } m_0\texttt{) = evalN Z + evalN } m_0$?

      Caso ind. n_0 = S n') HI) ¡$\texttt{evalN (addN n' } m_0\texttt{) = evalN n' + evalN } m_0$!

      TI) ¿$\texttt{evalN (addN (S n') } m_0\texttt{) = evalN (S n') + evalN } m_0$?

Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Relación entre manipulación y significado?
    - Una propiedad que vincule las funciones…

      Prop: ¿para todo **n**. para todo **m**.

      $$\texttt{evalN (addN n m)} = \texttt{evalN n} + \texttt{evalN m}?$$

      Dem: Sean n_0 y m_0 elementos totalmente definidos de **N**.

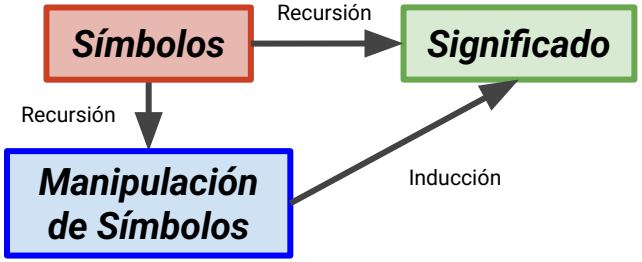
      Por ppio. de ind. en la estructura de n_0, se verá que

      Caso base n_0 = Z) ¿`evalN (addN Z` m_0`) = evalN Z + evalN` m_0?

      Caso ind. n_0 = S n') HI) ¡`evalN (addN n'` m_0`) = evalN n' + evalN` m_0!

      TI) ¿`evalN (addN (S n')` m_0`) = evalN (S n') + evalN` m_0?
    - La manipulación es ***coherente*** con el significado

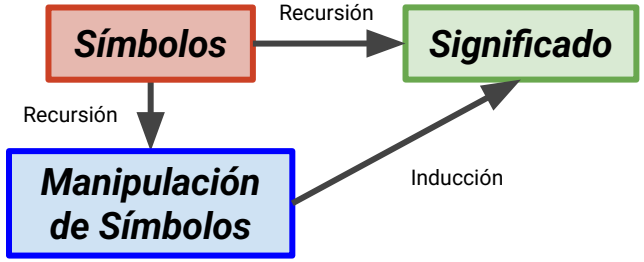
Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - Definición de *conjuntos de símbolos estructurados* por inducción
    - Ej. **`N`**
  - *Asignación de significado* mendiante funciones recursivas
    - Ej. **`evalN`**
  - *Tratamiento simbólico* mediante funciones recursivas
    - Ej. **`addN`**
  - *Coherencia* de manipulación y significado mediante inducción
    - Ej. para todo **n**. para todo **m**.

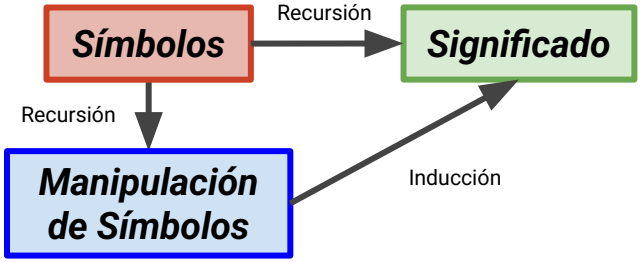
```
evalN (addN n m) = evalN n + evalN m
```

Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Y todo esto se puede hacer con significados más complejos, por ejemplo, un lenguaje de programación?

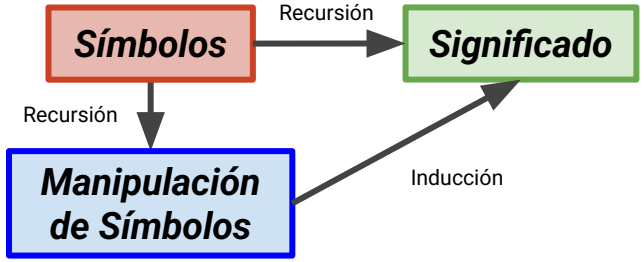
Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Y todo[illegible]s más comple[illegible]ramación?
  - Segura[illegible]



Representaciones de datos



- ¿Qué cosas interesantes se pueden representar?
  - ¿Y todo esto se puede hacer con significados más complejos, por ejemplo, un lenguaje de programación?
  - Seguramente se pueda descubrir más de esto en…

la Próxima Clase

Resumen

Resumen

- Principio de inducción estructural
  - Casos base
  - Casos inductivos (hipótesis inductivas y tesis inductiva)
- Listas y funciones sobre listas
  - Definición, recursión estructural sobre listas, demostración de propiedades sobre listas
- Representaciones de datos
  - Vínculos entre símbolos, significado y manipulación simbólica *usando solamente el lenguaje Haskell y las técnicas vistas*